Chapter 11
Graphics and Image Processing Libraries

Abstract In this chapter we discuss the Cairo graphics library. Cairo graphics library is used for device independent graphics rendering. Thereafter image formats such as PNG and JPEG are interfaced with the help of libraries libPNG and jpegLib, and these libraries are also discussed in this chapter. We conclude with a discussion of the GraphicsMagick and ImageMagick software.

Contents

11.1 Cairo: A Vector Drawing Library 201
11.2 Graphics File Formats 205
11.3 Conclusion 210

11.1 Cairo: A Vector Drawing Library

Cairo is a device independent vector drawing library which supports a number of backends including X11 (Xlib), Windows, PDF, PostScript, PNG, and MacOS (Quartz). The same rendering code (using Cairo drawing primitives) can generate the required output using the appropriate backend.

Cairo graphics supports rendering of fonts, arcs, points, polylines, as well as bitmap images. Drawing with Cairo involves the use of the `cairo_surface` and the `cairo_t` state. The surface models the backend, while the `cairo_t` state models the current drawing state machine. Attributes such as backing store of the surface, size, color space depth, number of visuals on the surface can be controlled using Cairo functions. The state machine attributes (including current color, pen type) can be changed on the `cairo_t` data-structure using Cairo functions. A simple example of using the X11 Xlib backend is shown in Listing 11.1.

```
// \file cairo_example.cpp
// \author Sandeep Koranne (C) 2010
// \description Use of Cairo graphics library
#include <iostream>           // for program IO
#include <cstdlib>            // for exit
```

S. Koranne, *Handbook of Open Source Tools*,
DOI 10.1007/978-1-4419-7719-9_11, © Springer Science+Business Media, LLC 2011

```
#include <cairo/cairo.h>      // Cairo library
#include <cairo/cairo-xlib.h> // X-lib surface for Cairo

static const unsigned int SIZE = 400;

static void EraseWindow( cairo_surface_t *surface ) {
  cairo_t *cr = cairo_create( surface );
  cairo_set_source_rgb( cr, 1.0, 1.0, 1.0 );
  cairo_paint( cr );
  cairo_destroy( cr );
}

static void DrawData( cairo_surface_t *surface ) {
  cairo_t *cr = cairo_create( surface );
  while( true ) {
    EraseWindow( surface );
    cairo_set_source_rgb( cr, 0.5, 0.1, 0.5 );
    cairo_rectangle( cr, rand()%SIZE,rand()%SIZE, 100, 100 );
    cairo_stroke( cr );
  }
  cairo_destroy( cr );
}

static void InitializeWindow( Display *dpy ) {
  cairo_surface_t *surface ;
  Drawable drawable;
  int screen = DefaultScreen( dpy );
  XSetWindowAttributes xwa;
  drawable = XCreateWindow( dpy, DefaultRootWindow( dpy ),
          100,100, SIZE, SIZE, 0,
          DefaultDepth( dpy, screen ),
          InputOutput,
          DefaultVisual( dpy, screen ),
          CWOverrideRedirect, &xwa );
  XMapWindow( dpy, drawable );
  surface = cairo_xlib_surface_create( dpy, drawable,
              DefaultVisual( dpy, screen ),
              SIZE, SIZE );
  cairo_xlib_surface_set_size( surface, SIZE, SIZE );
  DrawData( surface );

  EraseWindow( surface );
  XDestroyWindow( dpy, drawable );
}

int main( int argc, char *argv[] ) {
  Display *dpy = XOpenDisplay( NULL );
  if( dpy == NULL ) {
    std::cerr << "Unable to open X display...\n";
    exit(1);
  }
  InitializeWindow( dpy );

  std::cout << std::endl;
  XCloseDisplay( dpy );
  return (0);
}
```

Listing 11.1 Example of using Xlib with Cairo

The following functions are available for use when using Xlib surface with Cairo:

```
cairo_public cairo_surface_t *
cairo_xlib_surface_create (Display     *dpy,
         Drawable drawable,
         Visual      *visual,
         int    width,
```

```
          int    height);

cairo_public cairo_surface_t *
cairo_xlib_surface_create_for_bitmap (Display  *dpy,
               Pixmap  bitmap,
               Screen  *screen,
               int width,
               int height);

cairo_public void
cairo_xlib_surface_set_size (cairo_surface_t *surface,
           int              width,
           int              height);

cairo_public void
cairo_xlib_surface_set_drawable (cairo_surface_t *surface,
         Drawable   drawable,
         int              width,
         int              height);

cairo_public Display *
cairo_xlib_surface_get_display (cairo_surface_t *surface);

cairo_public Drawable
cairo_xlib_surface_get_drawable (cairo_surface_t *surface);

cairo_public Screen *
cairo_xlib_surface_get_screen (cairo_surface_t *surface);

cairo_public Visual *
cairo_xlib_surface_get_visual (cairo_surface_t *surface);

cairo_public int
cairo_xlib_surface_get_depth (cairo_surface_t *surface);

cairo_public int
cairo_xlib_surface_get_width (cairo_surface_t *surface);

cairo_public int
cairo_xlib_surface_get_height (cairo_surface_t *surface);
```

To draw using Cairo surfaces, a Cairo *context* `cairo_t` must be created using the Cairo surface as the target. The function `cairo_create()` is used for this purpose. The various attributes of the Cairo context can be changed using Cairo functions, e.g., the default line width can be changed using `cairo_set_line_width` function, and the *fill rule* can be set to either the *winding number*, or the even-odd overlap count using the `cairo_set_fill_rule` function. Even the *join types* of lines, their *stroke* (whether to use solid lines or dashes) can be set. In this regards, Cairo is similar to OpenGL (see Section 19.2) and GD (see Section 19.10).

Similar to OpenGL, a Cairo context can be given a matrix transform to be applied. The transform may contain rotation, translation, and scaling. The drawing primitives include (i) points, (ii) rectangles, (iii) paths, (iv) curves and arcs.

The advantage of using Cairo is manifest when the same rendering code has to generate multiple output formats such as PDF, PostScript, PNG, and SVG. Cairo provides functions to create *surfaces* for each of these backends. A page compositing program may use multiple backends of Cairo at the same time, Xlib for on-screen editing, PDF and PostScript for paper printing, and PNG and SVG for web

deployment. For example, the SVG (scalable vector graphics) backend functions are:

```
cairo_public cairo_surface_t *
cairo_svg_surface_create (const char   *filename,
        double  width_in_points,
        double  height_in_points);
```

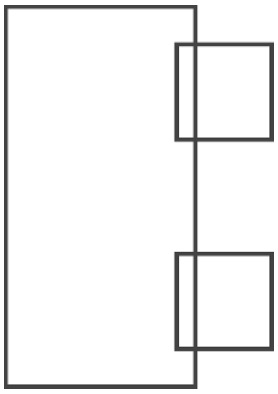


Fig. 11.1 Cairo graphics with SVG surface, example

An example of using SVG with Cairo is shown in Listing 11.2.

```
// \file cairo_svg.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of using SVG with Cairo
#include <cstdio>              // C FILE*
#include <cstdlib>             // exit
#include <iostream>            // program IO
#include <cairo/cairo.h>       // Cairo
#include <cairo/cairo-svg.h>   // SVG support
static unsigned int SIZE = 500;
static void DrawExample( const char* filename ) {
  cairo_surface_t *surface =
    cairo_svg_surface_create( filename, SIZE, SIZE );

  cairo_t *cr = cairo_create( surface );
  cairo_set_source_rgb( cr, 0.5, 0.1, 0.5 );
  #if 0
  cairo_translate( cr, 200, 100 );
  cairo_rotate( cr, 45.0 );
  #endif
  cairo_rectangle( cr, 10, 10, 100, 200 );
  cairo_rectangle( cr, 100, 30, 50, 50 );
  cairo_rectangle( cr, 100, 140, 50, 50 );

  cairo_stroke( cr );
  cairo_destroy( cr );
  cairo_surface_destroy( surface );
}

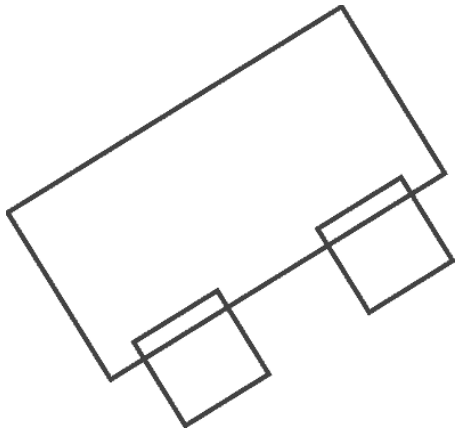
int main( int argc, char *argv [] ) {
  if( argc != 2 ) {
    std::cerr << "Usage: cairo_svg <filename>..\n";
    exit(1);
  }
  DrawExample( argv[1] );
  std::cout << std::endl;
  return (0);
```

```
}
```

Listing 11.2 Using SVG surface with Cairo

Compiling and running this program as:

Fig. 11.2 Example of using `cairo_rotate`



```
$g++ cairo_svg.cpp -lcairo
$./a.out a.svg
```

generates the SVG file as shown in Figure 11.1.

By adding `cairo_translate` and `cairo_rotate` to the Cairo context we can perform translation and rotation of the scene. An example is shown in Figure 11.2.

11.2 Graphics File Formats

Portable Network Graphics (PNG) is the modern loss-less image format which is widely used on the Internet. JPEG is loss-less and even though it is prevalent in Internet and amateur photography, its role in scientific image processing is thus limited only to preview generation. We concentrate on PNG as its design, and the software library are instructive in designing efficient file formats for two-dimensional bitmap graphics. The equivalent for vector images is Scalable Vector Graphics (SVG) (see Section 11.2.2) for more details.

11.2.1 libPNG: library for Portable Network Graphics

Portable Network Graphics (PNG) is a standard for loss-less representation of bit-mapped image data. PNG supports 24-bit and 32-bit RGBA color images (but does not support print color spaces such as CMYK). The PNG format uses the zlib

deflate compression (see Section 8.1) and achieves good compression. Since two-dimensional image data is often involved in scientific experiments; reading, writing, and otherwise manipulating PNG files is often required. We discuss the use of the `libPNG` library for this purpose. A more general purpose image manipulation library is discussed in the next section, see Section 11.2.3. Tools to programmatically generate PNG files are described in Chapter 19, see Section 19.10 and Section 19.11.

11.2.1.1 Format of a PNG file

PNG files are divided into *chunks*, which help in maintaining backwards compatibility. A PNG reader when confronted with a chunk which it cannot decode can simply skip the chunk (if it is not defined as critical). Each chunk is represented as:

1. Chunk length in bytes: this field is 4 bytes long,
2. Chunk type: also 4 bytes long. The case of the character in these 4 bytes denotes information that a decoder can use. This information includes whether the chunk is *critical*, *public*, *reserved*, and *safe to copy*,
3. Actual chunk data: length in bytes is given in the first field above,
4. CRC checksum: 4 bytes long.

The color depth of the image, compression, and transparency are also stored.

The compression used in PNG is divided into multiple phases. In the first phase a compression (using predictor functions) based on the value of neighboring pixels is performed. Then interlacing and *Adam7* interlacing (to quickly regenerate an approximation of the image during decoding) is done. This 7-pass scheme (Adam7) not only requires computation time, but also reduces compression since not all adjacent bytes of the images are compressed together. The raw byte stream comprising the chunk is then compressed using `deflate`, the standard ZLIB compression (see Section 8.1).

11.2.1.2 JPEG file format

The library for reading and writing JPEG files is `libjpeg` and it provides functions for reading (decompressing) and writing (compressing) JPEG image data from/to files. It provides a JPEG compress structure `jpeg_compress_struct` which can be used to write JPEG files, (part of the structure are shown) as shown in the listing below.

```
struct jpeg_compress_struct {
  /* partial structure see libjpeg.h */
  struct jpeg_destination_mgr * dest;
  JDIMENSION image_width; /* input image width */
  JDIMENSION image_height;  /* input image height */
  int input_components;
  J_COLOR_SPACE in_color_space; /* colorspace of input image */
  double input_gamma;   /* image gamma of input image */
  int data_precision;   /* bits of precision in image data */
  int num_components;   /* # of color components in JPEG image */
  J_COLOR_SPACE jpeg_color_space; /* colorspace of JPEG image */
```

```
  jpeg_component_info * comp_info;
  JHUFF_TBL * dc_huff_tbl_ptrs[NUM_HUFF_TBLS];
  JHUFF_TBL * ac_huff_tbl_ptrs[NUM_HUFF_TBLS];
};
```

Correspondingly, `jpeglib` also has a decompression structure for reading JPEG data as shown in the next listing (only some part of the structure are shown):

```
struct jpeg_decompress_struct {
  /* partial structure see libjpeg.h */
  struct jpeg_source_mgr * src;
  JDIMENSION image_width; /* nominal image width  */
  JDIMENSION image_height;  /* nominal image height */
  int num_components;   /* # of color components */
  J_COLOR_SPACE jpeg_color_space; /* colorspace of JPEG image */
  J_COLOR_SPACE out_color_space; /* colorspace for output */
  unsigned int scale_num, scale_denom;
  JDIMENSION output_width;  /* scaled image width */
  JDIMENSION output_height; /* scaled image height */
};
```

A list of function from the `jpeglib.h` header file is shown:

```
/* Destruction of JPEG compression objects */
 jpeg_destroy_compress JPP((j_compress_ptr cinfo));
 jpeg_destroy_decompress JPP((j_decompress_ptr cinfo));

 jpeg_stdio_dest JPP((j_compress_ptr cinfo, FILE * outfile));
 jpeg_stdio_src JPP((j_decompress_ptr cinfo, FILE * infile));

 jpeg_set_defaults JPP((j_compress_ptr cinfo));
 jpeg_set_colorspace JPP((j_compress_ptr cinfo,
                          J_COLOR_SPACE colorspace));
 jpeg_default_colorspace JPP((j_compress_ptr cinfo));
 jpeg_set_quality JPP((j_compress_ptr cinfo, int quality,
                       boolean force_baseline));
 jpeg_set_linear_quality JPP((j_compress_ptr cinfo,
                              int scale_factor,
                              boolean force_baseline));
 jpeg_add_quant_table JPP((j_compress_ptr cinfo, int which_tbl,
                           const unsigned int *basic_table,
                           int scale_factor,
                           boolean force_baseline));
 jpeg_quality_scaling JPP((int quality));
 jpeg_simple_progression JPP((j_compress_ptr cinfo));
 jpeg_suppress_tables JPP((j_compress_ptr cinfo,
                            boolean suppress));
 jpeg_alloc_quant_table JPP((j_common_ptr cinfo));
 jpeg_alloc_huff_table JPP((j_common_ptr cinfo));

/* Main entry points for compression */
 jpeg_start_compress JPP((j_compress_ptr cinfo,
                          boolean write_all_tables));
 jpeg_write_scanlines JPP((j_compress_ptr cinfo,
                           JSAMPARRAY scanlines,
                           JDIMENSION num_lines));
 jpeg_finish_compress JPP((j_compress_ptr cinfo));

 jpeg_write_raw_data JPP((j_compress_ptr cinfo,
                          JSAMPIMAGE data,
                          JDIMENSION num_lines));

/* Same, but piecemeal. */
 jpeg_write_m_header
 JPP((j_compress_ptr cinfo,
      int marker,
```

```
      unsigned int datalen));
 jpeg_write_m_byte
 JPP((j_compress_ptr cinfo, int val));

/* Alternate compression function:
   just write an abbreviated table file */
 jpeg_write_tables JPP((j_compress_ptr cinfo));

/* Decompression startup:
   read start of JPEG datastream to see what's there */
 jpeg_read_header JPP((j_decompress_ptr cinfo,
                       boolean require_image));

/* Main entry points for decompression */
 jpeg_start_decompress JPP((j_decompress_ptr cinfo));
 jpeg_read_scanlines JPP((j_decompress_ptr cinfo,
                          JSAMPARRAY scanlines,
                          JDIMENSION max_lines));
 jpeg_finish_decompress JPP((j_decompress_ptr cinfo));

/* Replaces jpeg_read_scanlines when reading raw downsampled data. */
 jpeg_read_raw_data JPP((j_decompress_ptr cinfo,
                         JSAMPIMAGE data,
                         JDIMENSION max_lines));

/* Additional entry points for buffered-image mode. */
 jpeg_has_multiple_scans JPP((j_decompress_ptr cinfo));
 jpeg_start_output JPP((j_decompress_ptr cinfo,
                        int scan_number));
 jpeg_finish_output JPP((j_decompress_ptr cinfo));
 jpeg_input_complete JPP((j_decompress_ptr cinfo));
 jpeg_new_colormap JPP((j_decompress_ptr cinfo));
 jpeg_consume_input JPP((j_decompress_ptr cinfo));
```

11.2.2 Scalable Vector Graphics (SVG)

In the above sections we have looked at image representation using PNG and JPEG. These file formats are designed for *bit-mapped* data. Another way to represent geometric information in images is using a *vector approach*, where the image is composed of well defined primitives. The primitives can be combined hierarchically, transformed (scaled and rotated, as well as translation). The advantage of vector graphics is: (i) faithful reproduction independent of scaling, (ii) much better compression for line art type drawings, (iii) programmatic generation, and (iv) elegant mathematical model underlying the drawing principles. Scalable Vector Graphics (SVG) is an international standard for representing vector image data using the XML format.

Since SVG files are XML files, they can be parsed using standard XML processing tools (see Section 9.4).

```
<?xml version="1.0" encoding="UTF-8"?>
<svg xmlns="http://www.w3.org/2000/svg"
           xmlns:xlink="http://www.w3.org/1999/xlink"
           width="500pt" height="500pt"
           viewBox="0 0 500 500" version="1.1">
<g id="surface1">
```

```
<path style="fill:none;stroke-width:2;stroke-linecap:butt;
             stroke-linejoin:miter;stroke:rgb(50%,10%,50%);
             stroke-opacity:1;stroke-miterlimit:10;"
             d="M 10.000546
 " transform="matrix(0.525322,0.850904,
                     -0.850904,0.525322,200,100)"/>
</g>
</svg>
```

The SVG specification defines the semantics of the XML elements; XML is used as the transmission medium and container. To properly interpret an SVG files, the semantics of the SVG primitives such as: (i) path, (ii) fill, (iii) width, (iv) stroke, (v) join-styles, etc. must be understood.

Thus, it is possible for an application to generate SVG directly, but most applications use a higher order abstraction. Generating XML from geometric primitives can be done using Cairo (see Section 11.1), and see Figure 11.2 for an example of using Cairo to generate a shape in SVG, then also perform a rotational transformation on the shape. Inkscape (see Section 19.8) uses SVG as its native file format.

11.2.3 GraphicsMagick and ImageMagick

GraphicsMagick is described as the "swiss army knife" of image processing. It provides tools and libraries which support reading and writing image of many formats including JPEG, PNG, and TIFF. Source code of GraphicsMagick is portable and recently has added OpenMP (see Section 12.2) support for parallel processing. GraphicsMagick has been used with large images (of gigapixel-size). GraphicsMagick is originally derived from ImageMagick but now is developed independently.

ImageMagick is an open-source software suite to create, edit, and compose bitmap images from a multitude of file formats including JPEG, PNG, and TIFF. ImageMagick can perform the following functions on files:

1. Format conversion: convert an image from one format to another,
2. Transform: resize, rotate, crop, and flip,
3. Transparency, Draw, and Decorate,
4. Special effects: blur, sharpen and tint,
5. Image identification,
6. Discrete Fourier Transform,
7. High dynamic range image,
8. Thread safe and implemented with OpenMP.

ImageMagick is mostly used with the command-line tool `convert` which has command-line options to perform many of the above tasks. Images used in this book were often processed with `convert` for resizing and format conversion.

11.3 Conclusion

In this chapter we discussed the Cairo graphics library, and presented example of its usage. Cairo is used for device independent graphics rendering in many open-source applications including Firefox. Image formats such as PNG and JPEG are interfaced with the help of libraries (libPNG and jpegLib). The Scalable Vector Graphics (SVG) file format (in XML) was explained with the help of an example, and we used Cairo to generate an SVG file. We concluded with a discussion of the GraphicsMagick and ImageMagick software for image manipulation on the command-line.

Part III
Parallel and System Programming